# for Loop

A **for loop** iterates over a sequence — a list, string, range, or any collection — executing the indented block once per item. Use it whenever you know ahead of time what you are looping over.

```
for variable in sequence:
    # block runs for each item
```

> *range(start, stop, step) is the most common sequence. range(5) gives 0,1,2,3,4. range(1,6) gives 1,2,3,4,5.*

## Key points

- The loop variable takes each value automatically — no manual indexing.
- Indentation (4 spaces) defines what is inside the loop.
- **break** exits early; **continue** skips to the next iteration.
- enumerate(seq) gives both the index and the value together.

---

### Example 1 — Count from 1 to 5

```
for i in range(1, 6):
    print(i)

# Output: 1  2  3  4  5
```

### Example 2 — Loop over a list

```
fruits = ['apple', 'banana', 'cherry']

for fruit in fruits:
    print('I like', fruit)

# I like apple
# I like banana
# I like cherry
```

### Example 3 — Sum of numbers 1 to 5

```
total = 0

for n in range(1, 6):
    total += n

print('Sum:', total)   # Sum: 15
```

### Example 4 — Loop over a string

```
for letter in 'Python':
    print(letter, end='-')

# Output: P-y-t-h-o-n-
```

### Example 5 — Multiplication table

```
for i in range(1, 11):
    print(f'3 x {i} = {3 * i}')

# 3 x 1 = 3
# 3 x 2 = 6
# ...up to 3 x 10 = 30
```

# while Loop

A **while loop** keeps running as long as its condition is **True**. Unlike a for loop, you do not need to know the number of iterations upfront — the loop decides at runtime.

```
while condition:
    # block runs while condition is True
    # make sure something changes so it eventually stops!
```

> *Always update something inside the loop so the condition eventually becomes False, otherwise you get an infinite loop.*

## Key points

- The condition is checked **before** each iteration.
- If the condition starts False, the body never runs.
- **break** exits the loop immediately.
- **continue** jumps to the next condition check.
- while True: with a break inside is a common pattern for menus.

### Example 1 — Countdown

```
count = 5

while count > 0:
    print(count)
    count -= 1

print('Blast off!')
# 5  4  3  2  1  Blast off!
```

### Example 2 — Guess the number

```
secret = 42
guess  = 0

while guess != secret:
    guess = int(input('Guess: '))

print('Correct!')
```

### Example 3 — Accumulate until limit

```
total, n = 0, 1

while total < 50:
    total += n
    n += 1

print('Total:', total)   # 55
```

### Example 4 — Password check with break

```
while True:
    pw = input('Password: ')
    if pw == 'python123':
        print('Access granted')
        break
    print('Wrong, try again')
```

**Example 5 — Print odd numbers with continue**

```
n = 0

while n < 10:
    n += 1
    if n % 2 == 0:   # skip evens
        continue
    print(n)

# 1  3  5  7  9
```

# match-case Statement

**match-case** (Python 3.10+) compares a value against a set of patterns and runs the first matching branch. It replaces long if-elif-else chains when you are checking the same variable.

```
match value:
    case pattern1:
        ...
    case pattern2:
        ...
    case _:        # wildcard — matches anything
        ...
```

> *Only the first matching case runs — there is no fall-through. Use | to match multiple values in one case.*

### Key points

- Requires Python 3.10 or newer.
- _ is the wildcard / default (like else).
- **case 1 | 2 | 3:** matches any of those values.
- Patterns can destructure lists and tuples automatically.
- No break needed — only one case ever runs.

---

### Example 1 — Day type

```
day = 'Monday'

match day:
    case 'Saturday' | 'Sunday':
        print('Weekend')
    case 'Monday':
        print('Start of the week')
    case _:
        print('Weekday')

# Start of the week
```

### Example 2 — HTTP status code

```
match status:
    case 200:
        print('OK')
    case 301 | 302:
        print('Redirect')
    case 404:
        print('Not Found')
    case 500:
        print('Server Error')
    case _:
        print('Unknown')
```

### Example 3 — Simple calculator

```
a, b, op = 10, 5, '+'

match op:
    case '+': print(a + b)   # 15
    case '-': print(a - b)   # 5
    case '*': print(a * b)   # 50
    case '/': print(a / b)   # 2.0
    case _:   print('Unknown operator')
```

### Example 4 — Tuple pattern — 2D point

```
point = (0, 5)

match point:
    case (0, 0): print('Origin')
    case (x, 0): print(f'X-axis at {x}')
    case (0, y): print(f'Y-axis at {y}')
    case (x, y): print(f'Point ({x}, {y})')

# Y-axis at 5
```

### Example 5 — Season from month

```
month = 7

match month:
    case 12 | 1 | 2:  print('Winter')
    case 3  | 4 | 5:  print('Spring')
    case 6  | 7 | 8:  print('Summer')
    case 9  | 10 | 11: print('Autumn')
    case _:           print('Invalid month')

# Summer
```